The Case for Single-Version Concurrency Control in the Cloud

Lam-Duy Nguyen
lamduy.nguyen@tum.de
Technische Universität München
Germany

Gabriel Haas
gabriel.haas@tum.de
Technische Universität München
Germany

Muhammad El-Hindi
muhammad.el-hindi@tum.de
Technische Universität München
Germany

Viktor Leis
leis@in.tum.de
Technische Universität München
Germany

Philip A. Bernstein
phil.bernstein@microsoft.com
Microsoft Research
USA

ABSTRACT

Multi-version concurrency control (MVCC) has been the dominant concurrency protocol in modern database management systems (DBMSs). However, cloud-native HTAP architectures shift many of MVCC's traditional advantages from the primary transaction engine to the deployment architecture. Now, transactional and analytical processing are decoupled into loosely synchronized engines, while cloud deployment primitives provide the workload isolation required for this design. In this paper, we challenge the status quo, arguing that single-version concurrency control (SVCC) is better suited to cloud transaction processing than MVCC. Through a systematic analysis of MVCC's limitations, we show that SVCC avoids these shortcomings through a more lightweight and efficient design. Beyond arguing for a lighter transaction processing, we use this SVCC effort as a blueprint to advocate for a broader rethinking of cloud-native architectures and to outline a research roadmap to shape next-generation DBMSs.

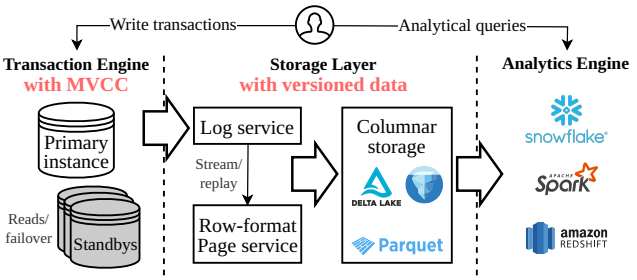


Figure 1: Major cloud data platforms process transactions in a single-writer transaction engine (left) and queries in a separate analytics engine (right), with updates flowing through the storage layer (middle). Versioning in both the transaction engine and storage layer creates unnecessary overhead. ⇨ shows the primary data path.

1 INTRODUCTION

MVCC has been the standard. As of 2026, the five top-ranked database management systems (DBMSs) in the DB-Engines ranking – Oracle, MySQL, SQL Server, PostgreSQL, and MongoDB – all implement multi-version concurrency control (MVCC). The appeal of MVCC over single-version concurrency control (SVCC) is that readers never block writers, so one engine can serve hybrid transactional and analytical processing (HTAP) workloads without a separate analytical system. To gain that capability, DBMSs accept the substantial complexity and space overhead of maintaining multiple versions.

The cloud changes design assumptions. The advent of cloud fundamentally changes the design objectives of database systems. Cloud platforms offer elastic compute resources and fast instance provisioning, enabling multiple database systems to be deployed and tailored to different workloads [18, 46, 72, 73]. These properties naturally encourage a separation between transactional and analytical deployments. The transaction side typically follows the single-writer architecture. In this design, a primary instance (Figure 1, left) handles write transactions, e.g., PostgreSQL, while analytical reads are served in a dedicated analytical engine (Figure 1, right) [2, 8, 25, 37, 39, 47, 63, 71]. This deployment is supported by row-oriented log and page services, with columnar storage populated through decoupled ETL pipelines or asynchronous transactional log streams, as shown in Figure 1 (middle). Making this deployment model practical at scale requires efficient replication between components; recent advances in log-based replication have addressed this challenge, significantly reducing latency and overhead [37, 44, 47]. Altogether, these developments enable workload separation at the deployment level, reducing the need for complex MVCC mechanisms to support long-running snapshots and HTAP within the primary instance.

MVCC adds overhead in single-writer cloud OLTP. Given the single-writer model in cloud OLTP systems, let us now reassess the role of MVCC. Under MVCC, each update creates a new physical version of a logical tuple, increasing memory pressure, storage footprint, and log volume. Many mechanisms that add CPU overhead, reduce cache efficiency, and performance instability are consequences of this design: garbage collection, version visibility checks, and version chain traversal [7, 43]. While these costs are justified in traditional systems that rely on MVCC for concurrent reads and writes, they are less important in single-writer cloud architectures. The cloud already provides workload isolation at the deployment

This paper is published under the Creative Commons Attribution 4.0 International (CC-BY 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution, provided that you attribute the original work to the authors and CIDR 2027. 17th Annual Conference on Innovative Data Systems Research (CIDR '27). January 24-27, Amsterdam, The Netherlands.

level through the storage layer or replicas, i.e., it conceptually resembles multi-versioning, making much of MVCC's benefits and associated overheads unnecessary on the primary instance.

SVCC is a better fit for cloud OLTP. In contrast, SVCC aligns naturally with the single-writer model. By keeping a single physical version per object, it reduces memory and storage overhead and simplifies the primary engine. As shown in Figure 1, workload separation is handled at the deployment level: analytics run on a separate engine, while the primary focuses on transactions. Additionally, if the primary instance can deliver high-performance serializability, the need to support weaker isolation levels inside the core engine for performance reasons vanishes, rendering one of MVCC's primary historical advantages unnecessary. Performance stability also improves because SVCC eliminates MVCC's garbage collection and version traversal, avoiding the overhead those mechanisms incur. As a result, the primary OLTP engine remains lightweight; most of the system complexity is now shifted to the deployment model, which is naturally enabled by cloud primitives.

Contributions. This paper re-examines concurrency control in the cloud era, focusing on the protocol used by the primary writer in a single-writer design. Specifically, we challenge the conventional wisdom that MVCC should be the default scheme for the primary writer, and instead argue that single-versioning is better suited for it. Our contributions are:

- **Section 2 – Revisiting MVCC:** We briefly discuss the historical background that led to the widespread adoption of MVCC. We then analyze MVCC in detail and expose the overheads MVCC incurs to achieve its advantages.
- **Section 3 – Cloud-centric analysis and SVCC positioning:** We analyze how cloud characteristics reshape concurrency control trade-offs. We then argue in detail that SVCC more closely aligns with cloud-native transaction processing than MVCC, offering lower overheads and reduced system complexity for OLTP.
- **Section 4 and Section 5 – Research directions:** We discuss related work and outline possible research opportunities to improve cloud-native HTAP architectures beyond the SVCC focus.

2 MVCC: THE GOOD AND THE UGLY

The importance of managing concurrent access in DBMSs was recognized early [24]. The key to that is the transaction abstraction, proposed by Jim Gray et. al. [34]. Jim Gray published many more papers on transaction processing [11, 32, 33, 35, 36], which emphasize the importance and benefits of transactions, and their atomicity, isolation (i.e., serializability) and durability (ACID) properties [13, 14, 31].

2.1 The Good

The early prevalence of single-version design. Many DBMSs soon followed and started integrating the transaction abstraction into their system. In many early implementations, a transaction overwrote the tuple value directly [40, 60], i.e., the system maintained only a single physical version of each record at any point in time. This single-version design was both natural and practical. Maintaining exactly one up-to-date copy of each tuple simplified storage management, recovery, and buffer management, making the approach straightforward to implement. Moreover, it aligned with the hardware constraints of the time – limited main memory and expensive magnetic disks – where minimizing redundancy was essential. Consequently, SVCC was the practical and economically sensible design choice in early database systems.

The rise of multi-version storage. In the mid-1980s, the price of magnetic disks declined to a point where data replication became economically feasible. This shift in storage economics fundamentally changed the design space of database systems. With storage capacity becoming less restrictive, DBMSs could now afford to maintain multiple physical versions of the same logical record [12]. Multi-version concurrency control preserves older versions of tuples instead of overwriting them in place [15, 16, 56]. Consequently, long-running read queries can access older tuple versions and are not blocked by concurrent write transactions, significantly improving read-write concurrency and overall system performance. Moreover, empirical studies show that many applications tolerate isolation levels weaker than full serializability [23, 53]. MVCC naturally supports such relaxed isolation models (e.g., snapshot isolation) while reducing contention between readers and writers. These practical advantages, combined with improved storage affordability, led to increased attention from database practitioners, which explains why MVCC was adopted by most modern DBMSs.

Primary advantage: Non-blocking reads. The main benefit of MVCC lies in its ability to decouple readers from writers. By preserving multiple physical versions of a tuple and associating them with visibility metadata (e.g., version timestamps), long-running queries can access a consistent snapshot of the database without conflicting with concurrent update transactions. Another commonly cited advantage of MVCC is *time-travel queries*, which allow users to access historical database states; however, this feature introduces substantial performance and storage overhead, leading PostgreSQL to deprecate and eventually remove its time-travel support in v6.2 [5]. Such snapshot-based reads offer fresher views of the database than offloading reads to replicas in cloud-native DBMS architectures, which typically incur millisecond-scale replication lag [8, 25, 63]. In workloads where reads dominate, this leads to substantial improvements in throughput and tail latency stability.

2.2 The Ugly

With the prevalence of MVCC, why do we argue for SVCC in cloud transaction processing? To answer, let us now discuss the trade-offs database designers must accept in exchange for MVCC's benefits.

Main trade-off: Complicated version management. The fundamental cost of MVCC is that maintaining multiple versions per tuple requires additional machinery beyond the concurrency control algorithm (e.g., two-phase locking). Unlike SVCC, MVCC treats every update as a version-creation event. This fundamentally changes the responsibilities of the storage engine: Instead of managing a single mutable state per key, the system must continuously create, track, validate, and eventually reclaim multiple coexisting versions. As a result, concurrency control is no longer isolated within the primary protocol. It becomes tightly integrated with storage layout, indexing data structures, and memory management. The complexity introduced by multi-versioning is therefore structural rather than incidental, as we illustrate in Figure 2 and examine in the following paragraphs.

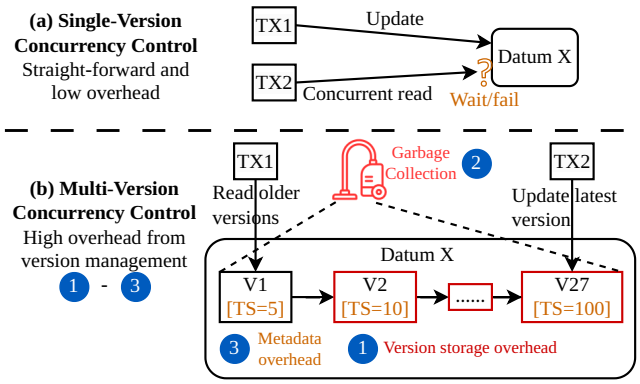


Figure 2: Unlike SVCC, MVCC incurs additional overheads and complexity due to version management.

Overhead 1: Version storage. Once updates produce new physical versions rather than overwriting in place, the system must decide where and how to store these versions for subsequent access. Common approaches include undo-log-based version chains, append-only delta stores, or separate version indexes [7, 43, 49, 66]. While these designs differ in locality and write amplification, none avoids the fundamental cost of retaining multiple versions.

Overhead 1 a: Version storage complexity. Specifically, version storage introduces unavoidable indirection. A lookup no longer focuses on only one record; instead, the engine must traverse the associated version chain to find a visible version. Under write-heavy workloads, these chains can grow, degrading cache locality and increasing pointer chasing costs. Alhomssi and Leis [7] shows that preventing unbounded version accumulation in MVCC requires a combination of carefully engineered mechanisms, including (1) post-commit visibility validation to detect isolation conflicts, (2) auxiliary data structures for managing old versions to enable efficient sweeping and also shorten version chains, and (3) in-row storage of recent versions to improve performance; all of these are non-trivial, significantly increasing system complexity. Studies in [43, 66] further demonstrate that preventing excessive MVCC version-chain growth is challenging and requires substantial modifications to the core execution engine.

Overhead 1 b: Version storage for large objects. Handling non-inline attributes (e.g., large VARCHAR, TEXT, JSON, or BLOB columns) further complicates version storage: The value of such attributes is typically not stored inline with the tuple; instead, the tuple contains only a pointer to separately allocated storage, introducing an additional level of indirection to access the actual object [52, 66]. Under MVCC, the DBMS may allow multiple versions to share the same external object; hence, reference counters are necessary to prevent excessive space consumption and ensure that values are reclaimed only when no version references them. This significantly complicates *garbage collection*, an already challenging component of MVCC as we will discuss next.

Overhead 2: Garbage collection. Because old versions may remain visible to active transactions, they cannot be reclaimed immediately. MVCC, therefore, requires a garbage collection mechanism to determine when a version becomes globally unreachable and can be safely removed. Garbage collection introduces several systematic trade-offs. First, determining global visibility information requires coordination, introducing additional synchronization overhead [7, 19]. Second, it requires background work that competes with transactions for CPU cycles, cache capacity, and I/O bandwidth. Third, garbage reclamation policies are inherently sensitive to workload characteristics: A single long-running transaction can delay cleanup indefinitely, leading to version accumulation and wasted space [7]. Conversely, aggressive reclamation increases bookkeeping overhead and interferes with transaction processing. Consequently, it is difficult to predict when garbage collection will occur and how much overhead it will incur, making it a persistent source of performance instability [19]. More importantly, garbage collection is not merely an optimization but a correctness requirement in MVCC. Without it, version chains grow unbounded, index entries accumulate, and storage consumption increases significantly over time. Thus, multi-versioning introduces a permanent maintenance responsibility that does not exist in SVCC.

Overhead 3: Tuple metadata and secondary index. Beyond version storage and garbage collection, MVCC imposes persistent space and runtime overheads due to tuple metadata and secondary indexes. Each tuple must store visibility metadata (e.g., timestamps and/or transaction IDs), leading to a typical header overhead of 16-32 bytes or more per record [7, 66]. For narrow schemas, this metadata can constitute a substantial fraction of the tuple size, reducing effective storage capacity and cache efficiency. Secondary index management is also more involved under MVCC, since both index keys and tuple pointers may change across versions. Different design choices exhibit different performance trade-offs across workloads, and index-only scans are generally infeasible unless visibility metadata is embedded directly into index entries, further increasing space overhead [66].

3 SVCC: THE BETTER FIT FOR THE CLOUD

Modern cloud data platforms comprise separate transactional and analytical engines connected through a disaggregated storage layer. In this section, we first identify the design principles that underpin these platforms and reshape concurrency-control requirements. We then show that these requirements render MVCC's primary overheads (c.f., Section 2.2) difficult to justify for the primary OLTP engine, making SVCC the more natural choice for cloud OLTP.

3.1 New Design Principles in Cloud DBMSs

Modern cloud data platforms follow three design principles that shape their high-level architecture and, in turn, their concurrency-control requirements.

Cost and resource efficiency as a necessity. In traditional on-premises deployments, databases ran on dedicated physical hardware, with the main goal being to maximize performance within fixed resource constraints. In contrast, cloud computing follows a pay-as-you-go model, where CPU, memory, storage, and network resources are billed based on usage. As a result, cost efficiency becomes a primary objective [27, 59, 61**?**]; any mechanism that consumes CPU time, memory, or storage must justify its cost. Consequently, designs that eliminate unnecessary overhead while maintaining high performance become increasingly attractive.

Deployment-level workload isolation. The elasticity of the cloud also enables new architectural patterns. On-demand instance provisioning [59, 64] and low-latency log-based replication [37, 39, 47] make it easy to run multiple instances of the same system. Instead of serving all workloads within a single engine, operators can deploy a dedicated instance for transaction processing and offload long-running analytical queries to replica instances or a separate engine. Workload isolation is thus provided at the deployment level rather than by the concurrency-control mechanism within an engine.

The single-writer architecture. In line with the principle of workload isolation, several major cloud DBMS vendors have adopted a *single-writer* architecture in their cloud OLTP systems, with examples including Amazon Aurora [63], Microsoft Socrates [8], Huawei Taurus [25], and Alibaba PolarDB [65]. In this architecture, one primary instance handles all writes while multiple replicas act as failover [2, 71]. The primary writer can therefore be optimized for short OLTP transactions, while read-intensive and analytical workloads execute elsewhere. The single-writer architecture thus reinforces the broader shift from engine-level to deployment-level workload isolation.

Implications for concurrency control. As discussed in Section 2, MVCC was designed to isolate different workloads within a single engine. These MVCC features were necessary when one database instance had to handle heterogeneous workloads. In cloud-native DBMSs, however, workload isolation can be achieved through replicas and elastic instances, with replica freshness typically within milliseconds [8, 25, 63]. Consequently, the primary OLTP engine no longer needs to execute long-running queries or provide cross-workload isolation. Its concurrency-control requirements are therefore substantially narrower. The relevant question is no longer how to support heterogeneous workloads within one engine, but which concurrency-control scheme best serves a dedicated OLTP writer. Given these architectural changes, is MVCC still the right concurrency control mechanism for the primary OLTP engine?

3.2 Why SVCC Fits Cloud OLTP

The cloud deployment model removes a principal motivation for MVCC in the primary OLTP engine: supporting heterogeneous workloads and long-running snapshots within a single engine. Once these workloads are isolated in dedicated analytical engines, the primary writer can focus on transactions. Under these narrower requirements, SVCC is not only viable but also a more natural foundation for cloud OLTP.

SVCC is cheaper at every level. As discussed in Section 2.2, MVCC introduces substantial engine-level complexity, resulting in runtime overheads that are not present in single-version designs. In particular, MVCC requires intricate mechanisms to manage multiple versions of both standard data types (e.g., numeric fields) and large objects. In contrast, SVCC avoids this additional machinery and relies only on the underlying concurrency control scheme, such as 2PL. Moreover, SVCC preserves a simple and intuitive model that remains consistent across all layers of the system.

Modern engines make serializability practical. To achieve good performance in the past, many applications deliberately accept weaker isolation levels – which are naturally enabled by MVCC-based designs [11, 49] – tolerating anomalies such as phantom reads or non-repeatable reads [23, 53]. However, we argue that applications do not have to make that trade-off anymore. State-of-the-art prototype engines can now achieve hundreds of thousands (or more) of transactions per second under strict serializability on the primary instance [28, 29, 39, 49, 74]. The throughput headroom is now large enough that most applications can adopt serializability by default, making SVCC the natural choice without sacrificing the correctness guarantees DBMSs previously had to compromise.

Table 1: Approximate complexity comparison. *Main* measures implementation effort for core components, while *Modif.* and *Tests* capture integration and testing effort.

Protocol	Lines of code (LoC)		
	Main	**Modif.**	**Tests**
Prototype implementations			
SVCC	303	45	665
MVCC	426	265	1325
Real implementations (AI-estimated)			
OSIC (Multi-VCC) [7]	~13K	–	–
PostgreSQL (Multi-VCC) [1]	~35K	–	–
MyISAM (Single-VCC) [4]	~2K	–	–

3.3 Complexity Evaluation

Setup. In this section, we use our own implementation experience as a concrete proxy to represent complexity. We use LeanStore [45], a modern storage engine that supports high throughput with μs-scale commit latency [50], and implement both protocols:

- **SVCC**: We implement traditional two-phase locking (2PL) with Wait-Die deadlock prevention. We use an open-sourced wait-die lock implementation combined with Intel TBB concurrent hash tables as an in-memory wait-die lock manager.
- **MVCC**: We implement Hyper-style MVCC [49]. In this protocol, a transaction modifies tuples in place, buffers their before-images in an undo log, and, at commit time, validates the writes of concurrently committed transactions against its read predicates.

In both implementations, we use key-level synchronization instead of range locking, e.g., predicate-based validation in Hyper [49], for simplicity of implementation. Although this modification permits phantom reads, it does not affect our comparison, since our focus is on complexity rather than performance.

Performance comparison. The performance characteristics of both protocols, as well as many other concurrency control designs (i.e., which workloads they are optimized for) are well-studied elsewhere [9, 10, 22, 38, 62, 66, 68, 69]; we do not revisit them in depth here. Our focus is on lightweight, clean prototypes that highlight the operational and implementation complexity of the two protocols. As a reference point, a lightweight TPC-C run (8 threads, serializable) achieves 127K txn/s for SVCC and 119K txn/s for MVCC in our prototype. This small performance gap reflects our prototype-oriented implementation: lock and version management both rely

on TBB concurrent hash tables that incur high overhead, and the short benchmark duration minimizes secondary effects such as garbage collection. We refer readers to the cited studies for comprehensive evaluations of those concurrency control protocols.

Reduced complexity in the prototype. Table 1 shows that SVCC comes with a consistently lower engineering effort across dimensions. SVCC requires 303 LoC for core implementation and 45 LoC for modifications, while MVCC needs 426 and 265 LoC, respectively, mainly due to version-chain integration with indexes. The gap is even larger in testing, where MVCC nearly doubles the code size (1325 vs. 665 LoC). Overall, MVCC incurs about 2–3× higher engineering cost. Furthermore, this difference is not just quantitative. MVCC requires developers to reason simultaneously about version chains, garbage collection, and their interactions with other subsystems. In contrast, SVCC provides a more uniform abstraction, simplifying both implementation and validation.

Complexity comparison of real-world implementations. Our prototype already suggests that SVCC requires substantially less implementation effort than MVCC (Table 1). To assess whether this trend extends to production systems, we compare a few representative implementations. For MVCC, we consider Ordered Snapshot Instant Commit (OSIC) [7] and PostgreSQL [1]; for single-version storage, we consider MyISAM [4]. Using ClaudeAI to estimate implementation complexity, we find that OSIC comprises approximately 13K LoC, PostgreSQL roughly 35K LoC for MVCC, whereas MyISAM requires only around 2K LoC. While we acknowledge this is an apples-to-oranges comparison – MyISAM uses table-level locking rather than row-level concurrency control – we argue that the statistics still provide a useful reference point for highlighting the implementation complexity of MVCC. Consistent with this observation, (1) a popular PostgreSQL internals book [58] devotes 108 pages (about 20% of its content) to transactions and MVCC, and (2) practitioners have noted that MVCC leaves its signatures across nearly every subsystem of PostgreSQL [6]. Together with our analysis, these observations suggest that cloud-native DBMSs can benefit from the simpler SVCC, which achieves both lower implementation complexity and higher performance in our study.

3.4 Lightweight Extensions

Building on the cloud-native architecture described in Figure 1, the primary instance now adopts SVCC as its concurrency control protocol (c.f., Figure 3). The lightweight design of SVCC intentionally scopes the primary engine to short transactions, which is the typical assumption for OLTP workloads. A transaction may, however, have a large write set (e.g., bulk loading), scan a large amount of data before performing an update, or remain active for an extended period. While such long-running transactions are a known challenge within the concurrency control community [7, 42], SVCC's simplicity leaves room for lightweight extensions to accommodate these exceptional cases. Below, we discuss several such extensions that we leave for future work.

1 Transaction limits. Long-running transactions are challenging under both single-version and multi-version storage. Depending on the protocol, such transactions retain locks, old versions, and/or the dependency graph for longer than typical transactions, thereby throttling the whole system. This also explains why recent

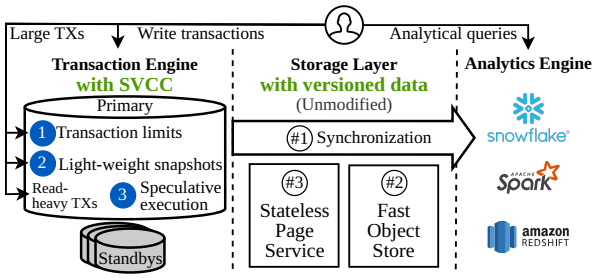


Figure 3: Our target architecture uses SVCC in the primary instance. Lightweight mechanisms are used to handle large transactions or read-heavy queries (i.e., #). Future directions for cloud HTAP are outlined as (#).

works propose to lock the entire database for long-running transactions [48, 51]. Production systems, therefore, commonly bound transaction size and duration. Amazon Aurora DSQL limits the data modified by a write transaction to 10 MiB and the duration of the transaction to five minutes. FoundationDB limits affected data to 10 MB and transaction duration to five seconds [20]. These restrictions reflect a pragmatic design choice: exceptional operations should be decomposed or handled separately rather than determining the common-case design of the transaction engine.

2 Lightweight snapshots. Another direction is to use a two-version extension on top of SVCC (e.g., 2V2PL [17], 2VTO [51]). However, these approaches still carry some of the overhead associated with MVCC. A cheaper alternative is to provide snapshots on demand: A transaction calls `BeginSnapshot()` to establish an empty auxiliary index – all overwritten tuples from concurrent transactions are inserted into this index, and `EndSnapshot()` discards this auxiliary index completely when that snapshot is no longer needed.

3 Speculative execution. Some transactions cannot be classified as read-only in advance – they may perform substantial computation before their first write. One possible approach is to begin such a transaction speculatively as read-only and upgrade it to a write transaction upon its first write. This upgrade operation needs to validate all earlier reads. If validation fails, the transaction can restart; alternatively, the upgrade may involve locking the whole database during the transition, similar to prior works [30, 51].

4 BEYOND SVCC: RETHINKING CLOUD HTAP

Our experience with SVCC suggests that many of the complexities in modern cloud-native DBMSs arise from legacy design assumptions rooted in traditional DBMS architectures. We therefore believe this is the time to step back and consider the broader system. Through this looking glass, we believe that the community has the opportunity to identify a leaner and more efficient architectural blueprint for cloud-native DBMSs. Many existing proposals [21, 26, 37, 55, 65, 67, 70] preserve these legacy assumptions, resulting in unnecessary complexity and overhead across the stack. Below, we outline several research directions that we believe are key to building the next generation of cloud-native DBMSs.

Direction (#1): HTAP synchronization. In traditional on-premise DBMSs, analytical queries rely on MVCC snapshots to achieve near-real-time visibility, as discussed in Section 2. In cloud-native HTAP architectures, however, analytical snapshots are maintained outside the transactional engine, i.e., in storage layer – Figure 3 (middle), making snapshot synchronization latency a primary optimization objective. Recent approaches reduce this lag by exposing recent updates through row-oriented page services [3] or log-caching layers [47]. Both approaches face the same issue: when the transactional engine produces a large volume of updates between the last synchronization point and the analytical snapshot timestamp, the analytical engine must merge all accumulated logs before answering queries, potentially negating the benefits of engines such as DuckDB or Raydon if the log gap is large (e.g., GBs). We therefore believe that efficient HTAP synchronization remains an open research problem. One direction is partitioning the log (and page) service by analytical predicates, allowing analytical engines to retrieve only relevant partitions and reducing unnecessary data movement.

Direction (#2): SSD-based object stores. Complementing software-level log partitioning, redesigning the underlying cloud object stores (e.g., Amazon S3) offers a hardware-aware path to reducing HTAP synchronization latency. Contemporary cloud object stores are inadequate not only for cloud-native HTAP systems but also for database workloads more broadly. This limitation stems from the fact that today's object stores largely inherit architectures designed nearly two decades ago and have not been fundamentally rearchitected to take advantage of modern hardware. Recent developments, such as S3 Express, reflect the growing demand for high-performance object storage; however, these remain incremental enhancements to the existing object-store abstraction and still incur access latencies that are too high for database workloads. We envision new SSD-based object-store primitives designed from the ground up for cloud-native DBMSs, preserving the scalability and durability properties while providing substantially lower access latency.

Direction (#3): Stateless page service. Another direction for optimizing cloud-native HTAP architectures is to make page materialization truly elastic. As analytical workloads become more intensive, the page service must scale elastically to sustain high request throughput. This, in turn, requires page service instances to join and leave as fast as possible. However, existing cloud page services (e.g., Databricks/Neon LTAP [3]) maintain dirty pages and asynchronously flush them to object storage, requiring coordination across page service instances. We propose a *truly stateless* page service based on the *log-is-the-data* paradigm. Durability, replication, and persistence are handled entirely by the log service, while buffer evictions and checkpointing append pages directly to the log service. As a result, page service instances become transient execution units that can be created, assigned to table predicates, and removed independently; this also enables page service deployment on serverless infrastructure. Thus, page materialization becomes a fully elastic compute layer rather than a stateful storage.

5 RELATED WORK AND SUMMARY

MVCC complexity. The complexity of MVCC protocols is well recognized. Wu et al. [66] show that MVCC incurs substantial overhead and complexity from version maintenance, garbage collection, and visibility checks. Complementary results in [62] demonstrate that MVCC only outperforms SVCC in limited cases (e.g., high-contention workloads). Overall, these findings suggest that MVCC's benefits are neither universal nor free, while introducing significant complexity and overhead.

Lock manager for storage-based DBMSs. Existing designs face a fundamental trade-off: they either assume that datasets fit entirely in memory and therefore do not support out-of-memory workloads [41, 57], or they scale poorly and/or incur excessive storage overhead [54]. As SVCC regains attention, a new design that is both scalable and space-efficient for out-of-memory workloads remains an important open challenge. A promising starting point is to adapt the per-page mapping table introduced by Umbra [30].

Summary. Our work is a first step toward the broader vision of how rethinking system components can reduce complexity while improving overall robustness. With this paper, we hope to inspire further research on re-architecting the cloud database stack and encourages a broader community effort to build a leaner, more robust foundation for the next generation of cloud databases.

REFERENCES

[1] 2023. PostgreSQL. https://github.com/postgres/postgres/tree/REL_15_3.
[2] 2026. Databricks Launches LTAP: The First Lake Transactional/Analytical Processing Architecture.
[3] 2026. From monolith to Lakebase to LTAP: rethinking the database from storage up.
[4] 2026. MySQL/MyISAM. https://github.com/mysql/mysql-server/tree/mysql-9.7.1/storage/myisam.
[5] 2026. PostgreSQL: Time Travel. https://www.postgresql.org/docs/6.3/c0503.htm.
[6] 2026. PostgreSQL's MVCC is bad. So is everyone else's. https://boringsql.com/posts/mvcc-bad-bad/.
[7] Adnan Alhomssi and Viktor Leis. 2023. Scalable and Robust Snapshot Isolation for High-Performance Storage Engines. *Proc. VLDB Endow.* 16, 6 (2023), 1426–1438.
[8] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Umar Farooq Minhas, Naveen Prakash, Vijendra Purohit, Hugh Qu, Chaitanya Sreenivas Ravella, Krystyna Reisteter, Sheetal Shrotri, Dixin Tang, and Vikram Wakade. 2019. Socrates: The New SQL Server in the Cloud. In *SIGMOD Conference*. ACM, 1743–1756.
[9] Tiemo Bang, Norman May, Ilia Petrov, and Carsten Binnig. 2020. The tale of 1000 Cores: an evaluation of concurrency control on real(ly) large multi-socket hardware. In *DaMoN*. ACM, 3:1–3:9.
[10] Tiemo Bang, Norman May, Ilia Petrov, and Carsten Binnig. 2022. The full story of 1000 cores. *VLDB J.* 31, 6 (2022), 1185–1213.
[11] Hal Berenson, Philip A. Bernstein, Jim Gray, Jim Melton, Elizabeth J. O'Neil, and Patrick E. O'Neil. 1995. A Critique of ANSI SQL Isolation Levels. In *SIGMOD Conference*. ACM Press, 1–10.
[12] Philip A. Bernstein. 2023. Eight Transaction Papers by Jim Gray.
[13] Philip A. Bernstein. 2025. Fifty Years of Transaction Processing Research. In *SIGMOD Conference Companion*. ACM, 1–2.
[14] Philip A. Bernstein. 2026. Fifty Years of Transaction Processing Research (extended). *CoRR* abs/2605.20466 (2026).
[15] Philip A. Bernstein and Nathan Goodman. 1980. Timestamp-Based Algorithms for Concurrency Control in Distributed Database Systems. In *VLDB*. IEEE Computer Society, 285–300.
[16] Philip A. Bernstein and Nathan Goodman. 1983. Multiversion Concurrency Control - Theory and Algorithms. *ACM Trans. Database Syst.* 8, 4 (1983), 465–483.
[17] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman. 1987. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley.
[18] Peter Boncz, Yannis Chronis, Jan Finis, Stefan Halfpap, Viktor Leis, Thomas Neumann, Anisoara Nica, Caetano Sauer, Knut Stolze, and Marcin Zukowski. 2023. SPA: Economical and Workload-Driven Indexing for Data Analytics in the Cloud. In *ICDE*. IEEE, 3740–3746.
[19] Jan Böttcher, Viktor Leis, Thomas Neumann, and Alfons Kemper. 2019. Scalable Garbage Collection for In-Memory MVCC Systems. *Proc. VLDB Endow.* 13, 2 (2019), 128–141.
[20] Marc Brooker, Marc Bowes, Mike Hershey, Zak van der Merwe, James Morle, and Matthys Strydom. 2026. Aurora DSQL: Scalable, Multi-Region OLTP. *CoRR* abs/2607.13276 (2026).

[21] Jianjun Chen, Yonghua Ding, Ye Liu, Fangshi Li, Li Zhang, Mingyi Zhang, Kui Wei, Lixun Cao, Dan Zou, Yang Liu, Lei Zhang, Rui Shi, Wei Ding, Kai Wu, Shangyu Luo, Jason Sun, and Yuming Liang. 2022. ByteHTAP: ByteDance's HTAP System with High Data Freshness and Strong Data Consistency. *Proc. VLDB Endow.* 15, 12 (2022), 3411–3424.

[22] Youmin Chen, Xiangyao Yu, Paraschos Koutris, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Jiwu Shu. 2022. Plor: General Transactions with Predictable, Low Tail Latency. In *SIGMOD Conference*. ACM, 19–33.

[23] Chaoyi Cheng, Mingzhe Han, Nuo Xu, Spyros Blanas, Michael D. Bond, and Yang Wang. 2023. Developer's Responsibility or Database's Responsibility? Rethinking Concurrency Control in Databases. In *CIDR*. www.cidrdb.org.

[24] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387.

[25] Alex Depoutovitch, Chong Chen, Jin Chen, Paul Larson, Shu Lin, Jack Ng, Wenlin Cui, Qiang Liu, Wei Huang, Yong Xiao, and Yongjun He. 2020. Taurus Database: How to be Fast, Available, and Frugal in the Cloud. In *SIGMOD Conference*. ACM, 1463–1478.

[26] Haowen Dong, Chao Zhang, Guoliang Li, and Huanchen Zhang. 2024. Cloud-Native Databases: A Survey. *IEEE Trans. Knowl. Data Eng.* 36, 12 (2024), 7772–7791.

[27] Kira Duwe, Angelos-Christos G. Anadiotis, Andrew Lamb, Lucas Lersch, Boaz Leskes, Daniel Ritter, and Pinar Tözün. 2025. The Five-Minute Rule for the Cloud: Caching in Analytics Systems. In *CIDR*. www.cidrdb.org.

[28] Tamer Eldeeb, Philip A. Bernstein, Asaf Cidon, and Junfeng Yang. 2024. Chablis: Fast and General Transactions in Geo-Distributed Systems. In *CIDR*. www.cidrdb.org.

[29] Tamer Eldeeb, Xincheng Xie, Philip A. Bernstein, Asaf Cidon, and Junfeng Yang. 2023. Chardonnay: Fast and General Datacenter Transactions for On-Disk Databases. In *OSDI*. USENIX Association, 343–360.

[30] Michael J. Freitag, Alfons Kemper, and Thomas Neumann. 2022. Memory-Optimized Multi-Version Concurrency Control for Disk-Based Database Systems. *Proc. VLDB Endow.* 15, 11 (2022), 2797–2810.

[31] Goetz Graefe (Ed.). 2019. *On Transactional Concurrency Control.* Morgan & Claypool Publishers.

[32] Jim Gray. 1980. A Transaction Model. In *ICALP (Lecture Notes in Computer Science, Vol. 85)*. Springer, 282–298.

[33] Jim Gray. 1981. The Transaction Concept: Virtues and Limitations (Invited Paper). In *VLDB*. IEEE Computer Society, 144–154.

[34] Jim Gray, Raymond A. Lorie, Gianfranco R. Putzolu, and Irving L. Traiger. 1975. Granularity of Locks in a Large Shared Data Base. In *VLDB*. ACM, 428–451.

[35] Jim Gray, Raymond A. Lorie, Gianfranco R. Putzolu, and Irving L. Traiger. 1976. Granularity of Locks and Degrees of Consistency in a Shared Data Base. In *IFIP Working Conference on Modelling in Data Base Management Systems*. North-Holland, 365–394.

[36] Jim Gray and Andreas Reuter. 1993. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.

[37] Tim Gubner, Rune Humborstad, and Manyi Lu. 2025. Freely Moving Between the OLTP and OLAP Worlds: Hermes - an High-Performance OLAP Accelerator for MySQL. *Proc. VLDB Endow.* 18, 12 (2025), 5113–5125.

[38] Zhihan Guo, Kan Wu, Cong Yan, and Xiangyao Yu. 2021. Releasing Locks As Early As You Can: Reducing Contention of Hotspots by Violating Two-Phase Locking. In *SIGMOD Conference*. ACM, 658–670.

[39] Michael Haubenschild and Viktor Leis. 2025. Oltp in the cloud: architectures, tradeoffs, and cost. *VLDB J.* 34, 4 (2025), 42.

[40] Gerald Held, Michael Stonebraker, and Eugene Wong. 1975. INGRES: A Relational Data Base System. In *AFIPS National Computer Conference (AFIPS Conference Proceedings, Vol. 44)*. AFIPS Press, 409–416.

[41] Hyungsoo Jung, Hyuck Han, Alan D. Fekete, Gernot Heiser, and Heon Young Yeom. 2014. A Scalable Lock Manager for Multicores. *ACM Trans. Database Syst.* 39, 4 (2014), 29:1–29:29.

[42] Jong-Bin Kim, Hyunsoo Cho, Kihwang Kim, Jaeseon Yu, Sooyong Kang, and Hyungsoo Jung. 2020. Long-lived Transactions Made Less Harmful. In *SIGMOD Conference*. ACM, 495–510.

[43] Jong-Bin Kim, Kihwang Kim, Hyunsoo Cho, Jaeseon Yu, Sooyong Kang, and Hyungsoo Jung. 2021. Rethink the Scan in MVCC Databases. In *SIGMOD Conference*. ACM, 938–950.

[44] Maximilian Kuschewski, Lam-Duy Nguyen, Matthias Jasny, Tobias Ziegler, Viktor Leis, and Muhammad El-Hindi. 2026. BtrLog: Low-Latency Logging for Cloud Database Systems. *Proc. VLDB Endow.* 19 (2026).

[45] Viktor Leis. 2024. LeanStore: A High-Performance Storage Engine for NVMe SSDs. *Proc. VLDB Endow.* 17 (2024), 4536–4545.

[46] Viktor Leis and Maximilian Kuschewski. 2021. Towards Cost-Optimal Query Processing in the Cloud. *Proc. VLDB Endow.* 14, 9 (2021), 1606–1612.

[47] Elena Milkai, Xiangyao Yu, and Jignesh M. Patel. 2025. Hermes: Off-the-Shelf Real-Time Transactional Analytics. *Proc. VLDB Endow.* 18, 8 (2025), 2334–2347.

[48] Thomas Neumann and Michael J. Freitag. 2020. Umbra: A Disk-Based System with In-Memory Performance. In *CIDR*. www.cidrdb.org.

[49] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. 2015. Fast Serializable Multi-Version Concurrency Control for Main-Memory Database Systems. In *SIGMOD Conference*. ACM, 677–689.

[50] Lam-Duy Nguyen, Adnan Alhomssi, Tobias Ziegler, and Viktor Leis. 2025. Moving on From Group Commit: Autonomous Commit Enables High Throughput and Low Latency on NVMe SSDs. *Proc. ACM Manag. Data* 3, 3, 191:1–191:24.

[51] Lam-Duy Nguyen, Sang Won Lee, and Beomseok Nam. 2022. In-Page Shadowing and Two-Version Timestamp Ordering for Mobile DBMSs. *Proc. VLDB Endow.* 15, 11 (2022), 2402–2414.

[52] Lam-Duy Nguyen and Viktor Leis. 2024. Why Files If You Have a DBMS?. In *ICDE*. IEEE, 3878–3892.

[53] Andrew Pavlo. 2017. What Are We Doing With Our Lives?: Nobody Cares About Our Concurrency Control Research. In *SIGMOD Conference*. ACM, 3.

[54] Dan R. K. Ports and Kevin Grittner. 2012. Serializable Snapshot Isolation in PostgreSQL. *Proc. VLDB Endow.* 5, 12 (2012), 1850–1861.

[55] Adam Prout, Szu-Po Wang, Joseph Victor, Zhou Sun, Yongzhu Li, Jack Chen, Evan Bergeron, Eric N. Hanson, Robert Walzer, Rodrigo Gomes, and Nikita Shamgunov. 2022. Cloud-Native Transactions and Analytics in SingleStore. In *SIGMOD Conference*. ACM, 2340–2352.

[56] David P. Reed. 1978. *Naming and synchronization in a decentralized computer system.* Ph. D. Dissertation. Massachusetts Institute of Technology, Cambridge, MA, USA.

[57] Kun Ren, Alexander Thomson, and Daniel J. Abadi. 2015. VLL: a lock manager redesign for main memory database systems. *VLDB J.* 24, 5 (2015), 681–705.

[58] Egor Rogov. 2023. PostgreSQL 14 Internals. *Postgres Professional* (2023).

[59] Till Steinert, Maximilian Kuschewski, and Viktor Leis. 2026. Cloudspecs: Cloud Hardware Evolution Through the Looking Glass. In *CIDR*. www.cidrdb.org.

[60] Michael Stonebraker, Eugene Wong, Peter Kreps, and Gerald Held. 1976. The Design and Implementation of INGRES. *ACM Trans. Database Syst.* 1, 3 (1976), 189–222.

[61] Junjay Tan, Thanaa M. Ghanem, Matthew Perron, Xiangyao Yu, Michael Stonebraker, David J. DeWitt, Marco Serafini, Ashraf Aboulnaga, and Tim Kraska. 2019. Choosing A Cloud DBMS: Architectures and Tradeoffs. *Proc. VLDB Endow.* 12, 12 (2019), 2170–2182.

[62] Takayuki Tanabe, Takashi Hoshino, Hideyuki Kawashima, and Osamu Tatebe. 2020. An Analysis of Concurrency Control Protocols for In-Memory Database with CCBench. *Proc. VLDB Endow.* 13, 13 (2020), 3531–3544.

[63] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *SIGMOD Conference*. ACM, 1041–1052.

[64] Midhul Vuppalapati, Justin Miron, Rachit Agarwal, Dan Truong, Ashish Motivala, and Thierry Cruanes. 2020. Building An Elastic Query Engine on Disaggregated Storage. In *NSDI*. USENIX Association, 449–462.

[65] Jianying Wang, Tongliang Li, Haoze Song, Xinjun Yang, Wenchao Zhou, Feifei Li, Baoyue Yan, Qianqian Wu, Yukun Liang, Chengjun Ying, Yujie Wang, Baokai Chen, Chang Cai, Yubin Ruan, Xiaoyi Weng, Shibin Chen, Liang Yin, Chengzhong Yang, Xin Cai, Hongyan Xing, Nanlong Yu, Xiaofei Chen, Dapeng Huang, and Jianling Sun. 2023. PolarDB-IMCI: A Cloud-Native HTAP Database System at Alibaba. *Proc. ACM Manag. Data* 1, 2 (2023), 199:1–199:25.

[66] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. 2017. An Empirical Evaluation of In-Memory Multi-Version Concurrency Control. *Proc. VLDB Endow.* 10, 7 (2017), 781–792.

[67] Jiacheng Yang, Ian Rae, Jun Xu, Jeff Shute, Zhan Yuan, Kelvin Lau, Qiang Zeng, Xi Zhao, Jun Ma, Ziyang Chen, Yuan Gao, Qilin Dong, Junxiong Zhou, Jeremy Wood, Goetz Graefe, Jeffrey F. Naughton, and John Cieslewicz. 2020. F1 Lightning: HTAP as a Service. *Proc. VLDB Endow.* 13, 12 (2020), 3313–3325.

[68] Chenhao Ye, Wuh-Chwen Hwang, Keren Chen, and Xiangyao Yu. 2023. Polaris: Enabling Transaction Priority in Optimistic Concurrency Control. *Proc. ACM Manag. Data* 1, 1 (2023), 44:1–44:24.

[69] Xiangyao Yu, George Bezerra, Andrew Pavlo, Srinivas Devadas, and Michael Stonebraker. 2014. Staring into the Abyss: An Evaluation of Concurrency Control with One Thousand Cores. *Proc. VLDB Endow.* 8, 3 (2014), 209–220.

[70] Matei Zaharia. 2025. Bringing the Operational and Analytical Worlds Together with Lakebase. *Proc. VLDB Endow.* 18, 12 (2025), 5539–5539.

[71] Chao Zhang, Guoliang Li, Jintao Zhang, Xinning Zhang, and Jianhua Feng. 2024. HTAP Databases: A Survey. *IEEE Trans. Knowl. Data Eng.* 36, 11 (2024), 6410–6429.

[72] Huanchen Zhang, Yihao Liu, and Jiaqi Yan. 2024. Cost-Intelligent Data Analytics in the Cloud. In *CIDR*. www.cidrdb.org.

[73] Xukang Zhang, Huanchen Zhang, and Xiaofeng Meng. 2025. Accordion: Balancing Performance and Cost in Cloud-Native Data Analysis with Intra-Query Runtime Elasticity. *Proc. VLDB Endow.* 18, 12 (2025), 5307–5310.

[74] Michael Zinsmeister, Lam-Duy Nguyen, Viktor Leis, and Thomas Neumann. 2026. Predictive Translation: High-Performance Buffer Management Without the Trade-Offs. In *SIGMOD Conference*. ACM.